



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
Second Edition, O'Reilly Media 2020

Design Pattern 1 Strategy and Observer

Francesco Guerra
francesco.guerra@unimore.it

Motivation

▶ OO Principle

▶ Encapsulation

- ▶ Binding the data with the code that manipulates it.
- ▶ It keeps the data and the code safe from external interference

▶ Polymorphism

- ▶ Polymorphism means to process objects differently based on their data type
- ▶ One method with multiple implementation, for a certain class of action. And which implementation to be used is decided at runtime depending upon the situation i.e., data type of the object
 - ❑ ad hoc polymorphism (**“method overloading”**)
 - ❑ inclusion polymorphism (**override o “Inclusion polymorphism”**)
 - ❑ Changing the implementation and/or return type in an extended class
 - ❑ parametric polymorphism (o generic programming)
 - ❑ The idea is to allow type (Integer, String, ... etc and user defined types) to be a parameter to methods, classes and interfaces. For example, classes like HashSet, ArrayList, HashMap, etc use generics very well.

Motivation

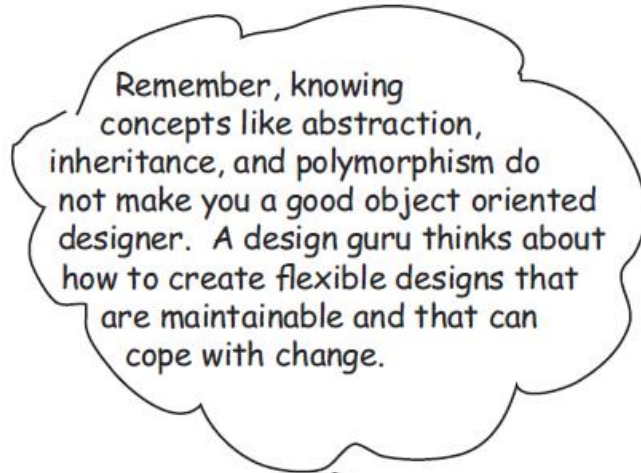
► Inheritance

- Inheritance is the mechanism by which an object acquires the some/all properties of another object
- It supports the concept of hierarchical classification

Example

- ▶ Create a Person class with the attributes name, age, and friends (a list of friends), and a method say.
- ▶ Create an Italian class that makes a person say “Ciao gente!” through the method parla.
- ▶ Create an English class that makes a person say “Hello World!” through the method speak.
- ▶ Create a Test class that instantiates Frank, an English person, and Francesco, an Italian person (Francesco is 29 years old and his name must be provided in the class constructor).
 - ▶ Frank is a friend of Francesco, but Francesco is not a friend of Frank.
 - ▶ Both persons want to say something through the common method say.
- ▶ Create a function that, given a list of people, prints each person’s name and the names of their pets.
- ▶ The implementation must use encapsulation, polymorphism (all three forms), and inheritance.

Is this enough to be a good designer?

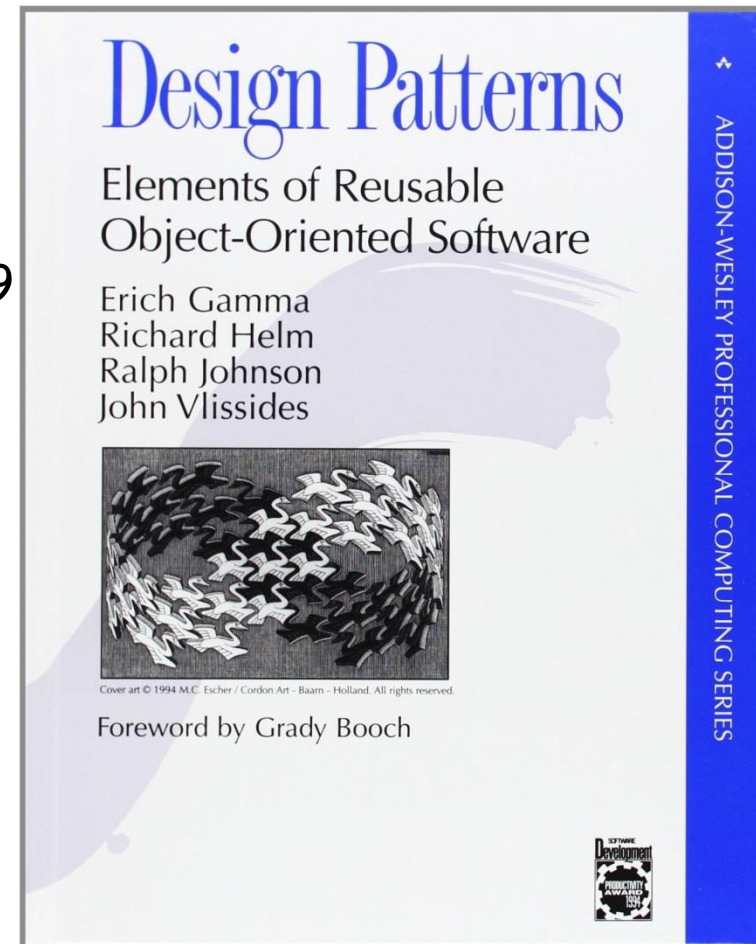


Design Patterns help you to cope with this issue

- By providing a set of solutions to common Design issues
- By providing you a vocabulary to communicate with other programmers and developers
- You can speed up your expertise by taking advantages by the experience of other developers

The history of the Design Pattern

- ▶ The work published by the architect **Christopher Alexander** introduced the term “pattern” in 1977
- ▶ **Erich Gamma**, Ph.D. Thesis, 1991
- ▶ James Coplien, Advanced C++ Idioms, 1989
- ▶ Gamma, Helm, Johnson, Vlissides **Gang of Four - GoF**, Design Patterns: Elements of Reusable Object-Oriented Software, 1991
- ▶ Buschmann, Meunier, Rohnert, Sommerland, Stal, Pattern-Oriented Software Architecture: A System of Patterns, 1996
- ▶ Conferenze PLoP, started in 1994



Definitions

- ▶ ...describes a **problem** which occurs over and over again in our environment, and then describes the core of the **solution** to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice,
C. Alexander
- ▶ ...the **abstraction** from a concrete form which keeps recurring in specific non-arbitrary contexts,
D. Riehle
- ▶ ...both a thing and the **instructions** for making the thing,
J. Coplien

Features

1. The **pattern name** is a handle we can use to describe a design problem, its solutions, and consequences.
 - ▶ Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction.
2. The **problem** describes when to apply the pattern.
 - ▶ It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects.
3. The **solution** describes the elements that make up the design, their relationships, responsibilities, and collaborations.
 - ▶ The pattern provides an abstract description of a design problem and how a general arrangement of elements classes and objects in our case solves it.
4. The **consequences** are the results and trade-offs of applying the pattern.
 - ▶ they are critical for evaluating design alternatives and for understanding the costs and benefits of applying the pattern.

Describing Design Patterns - GoF

▶ **Pattern Name and Classification**

- ▶ The pattern's name conveys the essence of the pattern succinctly. A good name is vital, because it will become part of your design vocabulary.

▶ **Intent**

- ▶ What does the design pattern do? What is its rationale and intent? What particular design issue or problem does it address?

▶ **Also Known As**

- ▶ Other well-known names for the pattern, if any.

▶ **Motivation**

- ▶ A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem.

▶ **Applicability**

- ▶ What are the situations in which the design pattern can be applied? What are examples of poor designs that the pattern can address? How can you recognize these situations?

Describing Design Patterns - GoF (2)

▶ **Structure**

- ▶ A graphical representation of the classes in the pattern.

▶ **Participants**

- ▶ The classes and/or objects participating in the design pattern and their responsibilities.

▶ **Collaborations**

- ▶ How the participants collaborate to carry out their responsibilities.

▶ **Consequences**

- ▶ How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?

▶ **Implementation**

- ▶ What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?

Describing Design Patterns- GoF (3)

- ▶ **Sample Code**

- ▶ Code fragments that illustrate how you might implement the pattern in C++ or Smalltalk.

- ▶ **Known Uses**

- ▶ Examples of the pattern found in real systems.

- ▶ **Related Patterns**

- ▶ What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

Organizing the Catalog- GoF

Scope		Purpose		
		Creational	Structural	Behavioral
	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

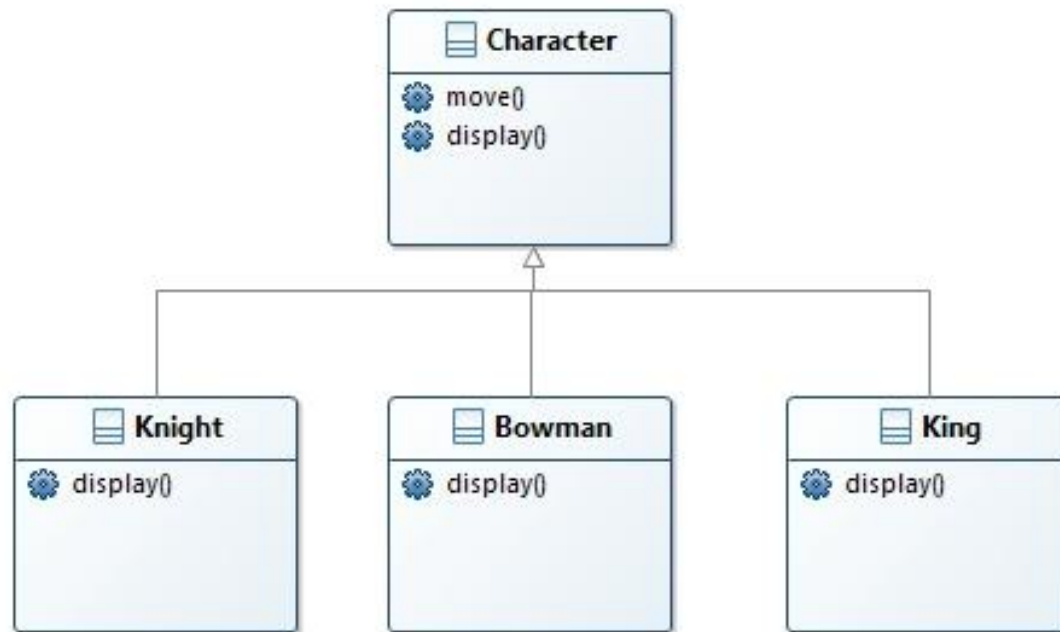
Organizing the Catalog- GoF (2)

- ▶ **Class Patterns**
 - ▶ Describe how relationships between classes are defined via inheritance.
 - ▶ Relationships are established at compile time.
- ▶ **Object Patterns**
 - ▶ Describe relationships between objects, primarily via composition.
 - ▶ Relationships are created at runtime and are more dynamic and flexible.
- ▶ **Creational Patterns**
 - ▶ Deal with object creation mechanisms.
 - ▶ Help make a system independent of how its objects are created, composed, and represented.
- ▶ **Structural Patterns**
 - ▶ Focus on how classes and objects are composed to form larger structures.
 - ▶ Make it easier to change the structure of a system without affecting client code.
- ▶ **Behavioral Patterns**
 - ▶ Deal with object interaction and responsibility.
 - ▶ Define how objects communicate and distribute behavior.

Design Pattern 1: Strategy

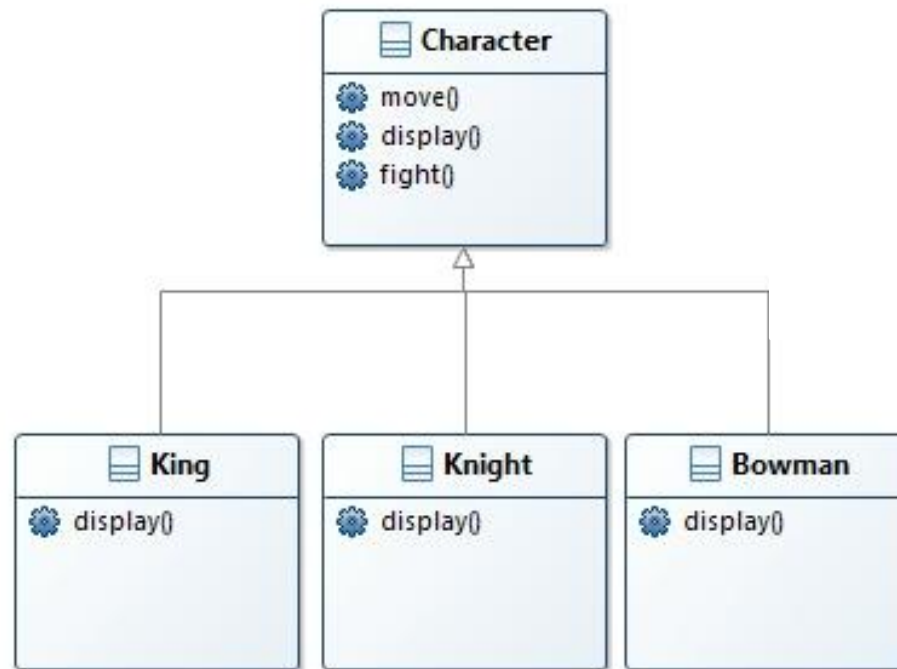
- ▶ Motivating example
 - ▶ Our company makes role-playing games.
 - ▶ The initial system designer adopted standard OO modeling techniques for representing the various kinds of characters

Overriding allows the developer to provide a specific implementation for the methods



Motivating example issues with inheritance

- ▶ We have to introduce a new method in the characters. Now they can “fight”



Not all classes have to fight!!!
Wizards cast spells, Unicorns do nothing!

Motivating example Solution

- ▶ I could override methods in the specific classes that don't need the default behavior.
- ▶ But...
 - ▶ What happens when I introduce a new class, e.g., Clerics?
 - ▶ What if I need to add a class for Trolls?
 - ▶ How can I implement different kinds of fight and different kinds of casting spells?
 - ▶ What if the fight method needs to be changed in all 50 classes implementing it?
 - ▶ What if every 4-6 months I need to add new classes and modify existing ones?
 - ▶ Key Question
 - ▶ Is it possible to model the application so that changes can be made with minimal impact on existing code?

Design Principle

- ▶ The one constant in software development:
 - ▶ No matter how well you design an application, over time an application must grow and change or it will die
- ▶ Design Principle
 - Identify the aspects of your application that vary and separate them from what stays the same.
 - ▶ Take the parts that vary and "encapsulate" them
 - ▶ This way you can alter the parts that vary without affecting those that don't.
 - ▶ It forms the basis for almost every design pattern.
 - ▶ All patterns provide a way to let some part of a system vary independently of all other parts.

Design Principle (2)

- ▶ **Design Principle**
Program to an interface, not an implementation.
- ▶ We'll use an interface to represent the behavior
 - ▶ Each implementation of a behavior will implement one of those interfaces -- FightBehaviour
 - ▶ To exploit polymorphism by programming to a supertype so that the actual runtime object isn't locked into the code.

Programming to an interface

- ▶ Programming to an implementation

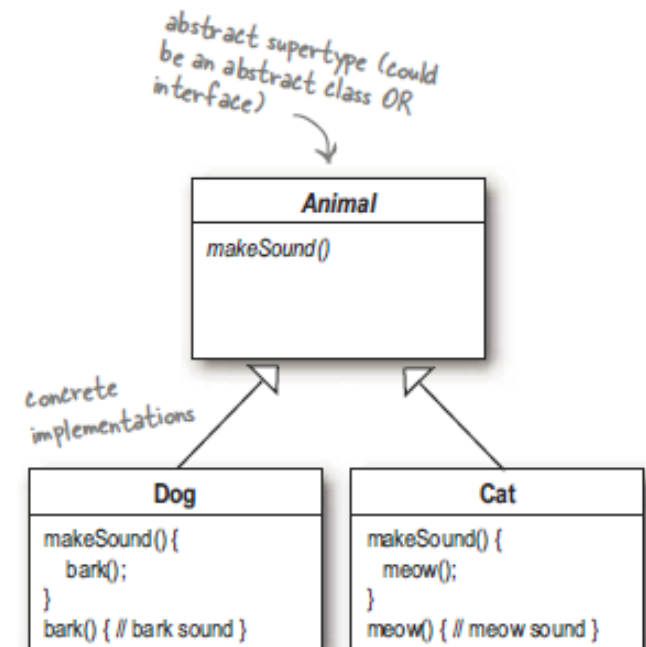
```
Dog d = new Dog();
```

```
d.bark();
```

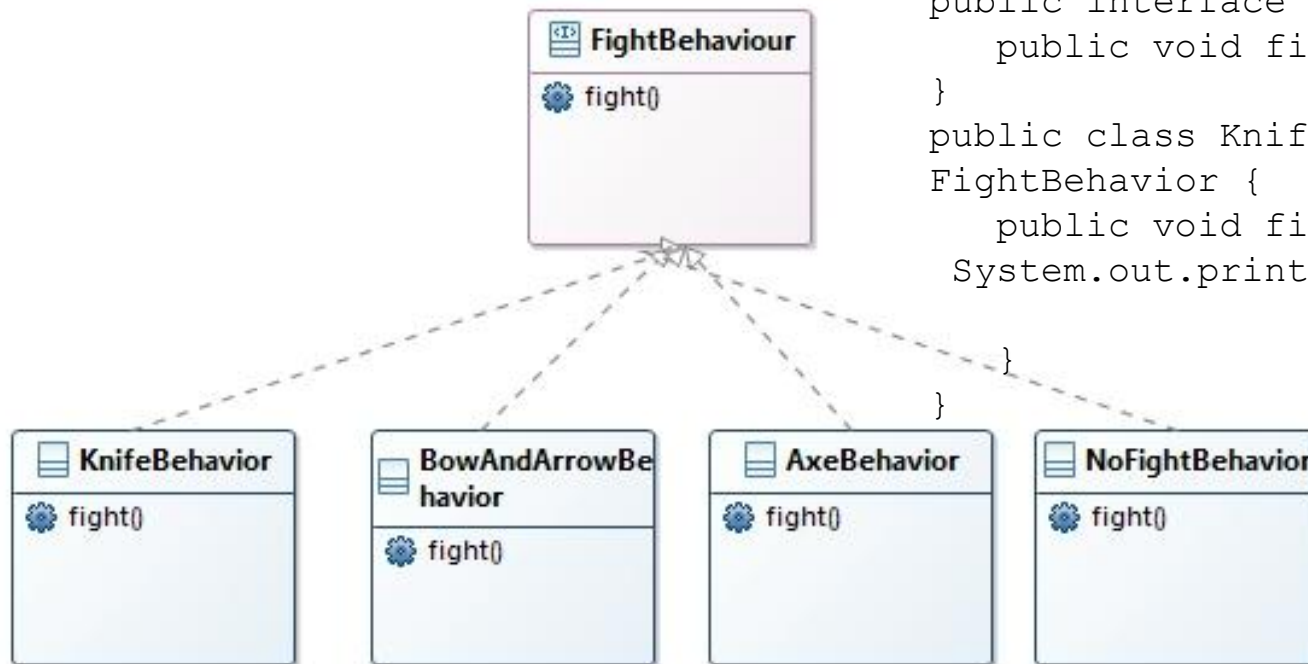
- ▶ Programming to an interface

```
Animal animal = new Dog();
```

```
animal.makeSound();
```



Implementing FightBehavior

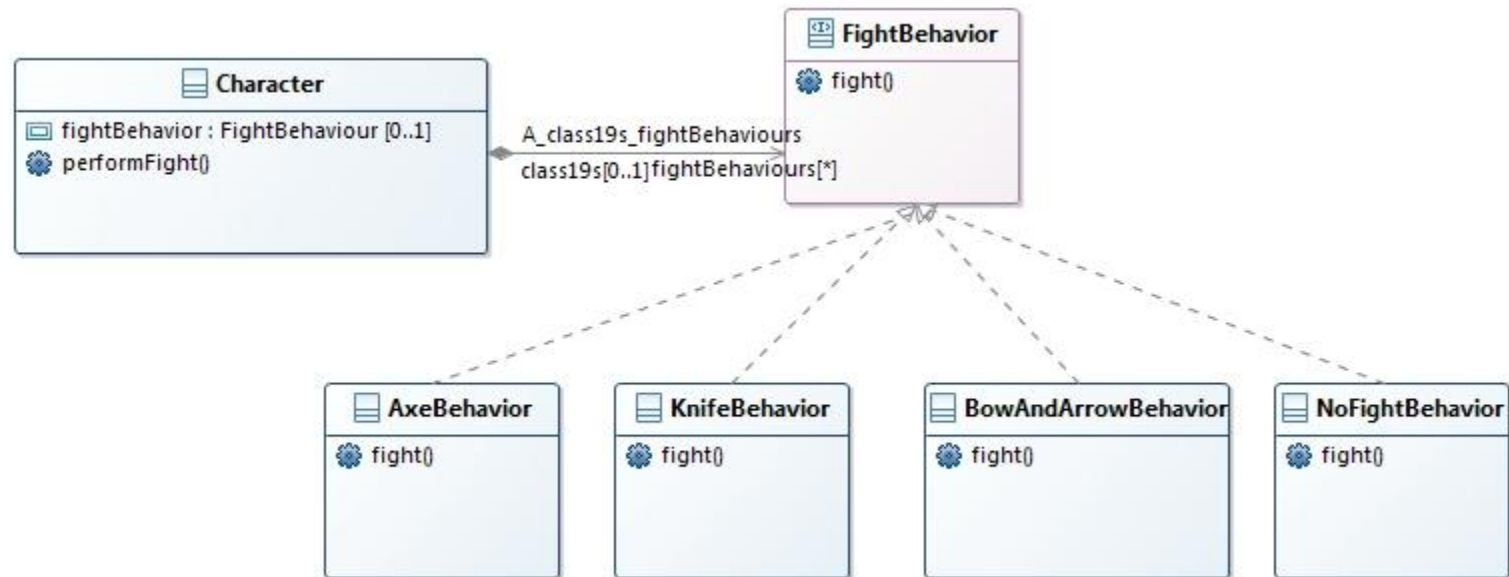


```
public interface FightBehavior {
    public void fight();
}

public class KnifeBehavior implements
FightBehavior {
    public void fight() {
        System.out.println("I'm fighting with
a knife!!");
    }
}
```

- ▶ The method `fight()` implements the specific task for the specific behavior.
- ▶ With this design, other types of objects can reuse the behaviors because they are no longer hidden away in the Character classes
- ▶ We can add new behaviors without modifying any of our existing behavior classes or touching any of the Character classes that use fight behaviors.

Integrating the FightBehavior

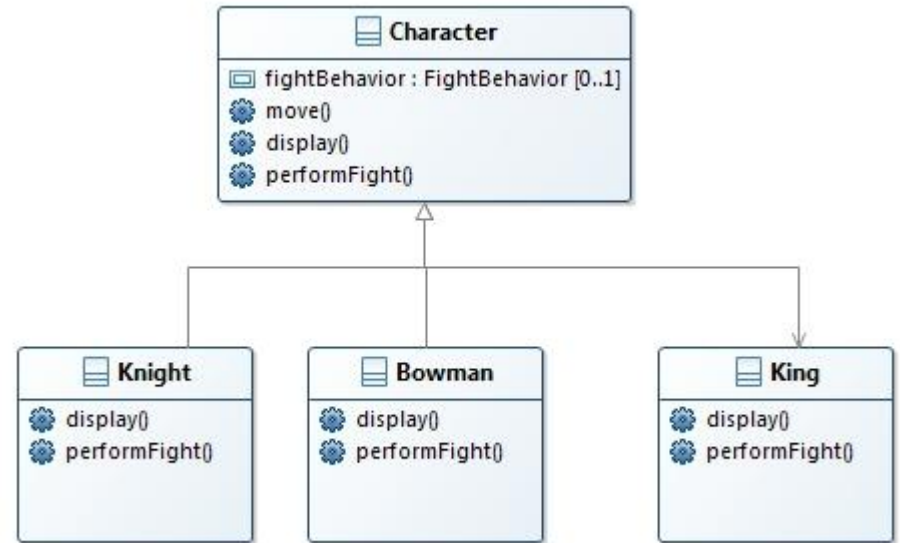


```
public class Character{
    FightBehavior fightBehavior;
    // more
    public void performFight() {
        fightBehavior.fight();
    }
}
```

- ▶ Defining Character we don't care about the specific behavior, we delegate the interface

Implementing a specific Character

```
public class Character{  
    FightBehavior fightBehavior;  
    // more  
    public void performFight() {  
        fightBehavior.fight();  
    }  
}
```



```
public class King extends Character {  
    public King() {  
        fightBehavior = new KnifeBehavior();  
    }  
    public void display() {  
        System.out.println("I'm a king");  
    }  
}
```

Implementation

```
public abstract class Character{
    FightBehavior fightBehavior;
    // more
    public void Character() {
    }
    public void performFight() {
        fightBehavior.fight();
    }
    public abstract void display();
    public void move() {
        System.out.println("I'm moving!");
    }
}

public class King extends Character {
    public King() {
        fightBehavior = new
        KnifeBehavior();
    }
    public void display() {
        System.out.println("I'm a king");
    }
}
```

```
public interface FightBehavior {
    public void fight();
}

public class KnifeBehavior implements
FightBehavior {
    public void fight() {
        System.out.println("I'm fighting with
                            a knife!!");
    }
}

public class Simulator{
    public static void main(String[]
args) {
        Character lion= new King();
        lion.performFight();
    }
}
```



Issue in this design

- ▶ We are modeling the behavior of the character via its constructor
 - ▶ **Program to an interface, not an implementation** is a design principle!
- ▶ Still, notice that while we are setting the behaviors to concrete classes, we could easily change that at runtime.
- ▶ So, we still have a lot of flexibility here, but we're doing a poor job of initializing the instance variables in a flexible way.
 - ▶ Since the `fightBehavior` instance variable is an interface type, we could dynamically assign a different `FightBehavior` implementation class at runtime.
 - ▶ (this is the advantages of polymorphism)

Setting behavior dynamically

```
public abstract class Character{
    FightBehavior fightBehavior;
    //...
    public void setFightBehavior(FightBehavior fb){
        fightBehavior= fb;
    }
}

public class King extends Character {
    public King() {
        fightBehavior = new KnifeBehavior();
    }
}

public class Simulator{
    public static void main(String[] args) {
        Character lion= new King();
        lion.performFight();
        lion.setFightBehavior(new AxeBehavior());
        lion.performFight()
    }
}
```

The Big Picture

- ▶ Instead of thinking of the character behaviors as a set of behaviors, we'll start thinking of them as a family of algorithms.
- ▶ In the simulator, the algorithms represent things a Character would do (different ways of fighting), but we could just as easily use the same techniques for other scenarios
 - ▶ a set of classes that implement the ways to compute state sales tax by different states.
- ▶ When you put two classes together like we did you're using composition (HAS-A instead of IS-A relationship).
 - ▶ Instead of inheriting their behavior, the characters get their behavior by being composed with the right behavior object.
- ▶ **Design Principle**
Favor composition over inheritance.

Key Points

- ▶ Knowing the OO basics does not make you a good OO designer.
- ▶ Good OO designs are reusable, extensible, and maintainable.
- ▶ Patterns show you how to build systems with good OO design qualities. □
 - ▶ Patterns are proven object-oriented experience.
- ▶ Patterns don't give you code, they give you general solutions to design problems. You apply them to your specific application.
- ▶ Patterns aren't invented, they are discovered.
- ▶ Most patterns and principles address issues of change in software. □
- ▶ Most patterns allow some part of a system to vary independently of all other parts.
- ▶ We often try to take what varies in a system and encapsulate it.
- ▶ Patterns provide a shared language that can maximize the value of your communication with other developers

Strategy Pattern Description - GoF structure (partial)

▶ Intent

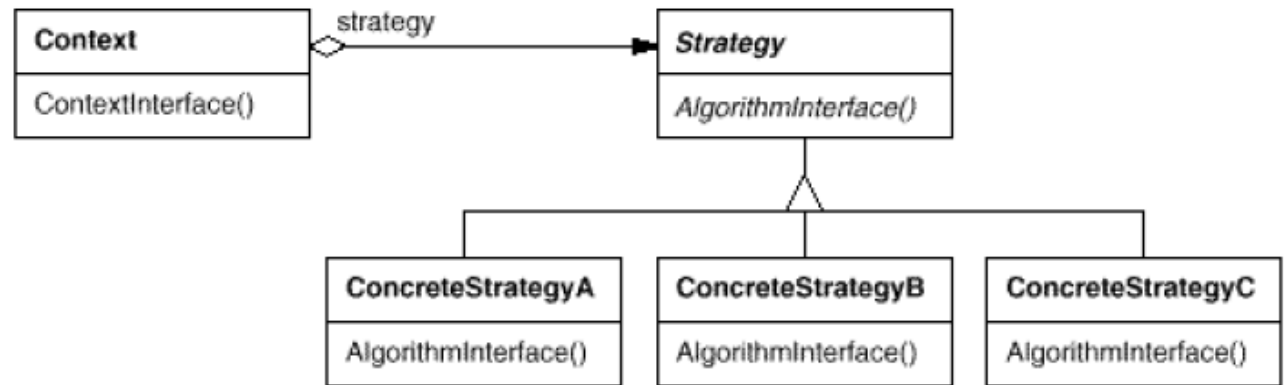
- ▶ Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

▶ Also Known As

- ▶ Policy

▶ Applicability

- ▶ Use the Strategy pattern when
 - ▶ many related classes differ only in their behavior. Strategy provides a way to configure a class with one of many behaviors.
 - ▶ you need different variants of an algorithm.
 - ▶ an algorithm uses data that clients shouldn't know about. Use the Strategy pattern to avoid exposing complex, algorithm-specific data structures
 - ▶ a class defines many behaviors, and these appear as multiple conditional statements in its operations. Instead of many conditionals, move related conditional branches into their own Strategy class.



► Structure

► Participants

► Strategy

- declares an interface common to all supported algorithms. Context uses this interface to call the algorithm defined by a ConcreteStrategy.

► ConcreteStrategy

- implements the algorithm using the Strategy interface.

► Context

- is configured with a ConcreteStrategy object.
- maintains a reference to a Strategy object.
- may define an interface that lets Strategy access its data.

Strategy Pattern Description(3)

► Consequences

- *Families of related algorithms.*
Hierarchies of Strategy classes define a family of algorithms or behaviors for contexts to reuse. Inheritance can help factor out common functionality of the algorithms.
- *An alternative to subclassing.*
Inheritance offers another way to support a variety of algorithms or behaviors.
 - You can subclass a Context class directly to give it different behaviors. But this hard-wires the behavior into Context.
 - It mixes the algorithm implementation with Context's, making Context harder to understand, maintain, and extend.
 - You wind up with many related classes whose only difference is the algorithm or behavior they employ.
 - Encapsulating the algorithm in separate Strategy classes lets you vary the algorithm independently of its context, making it easier to switch, understand, and extend.
- *Strategies eliminate conditional statements.*
The Strategy pattern offers an alternative to conditional statements for selecting desired behavior.

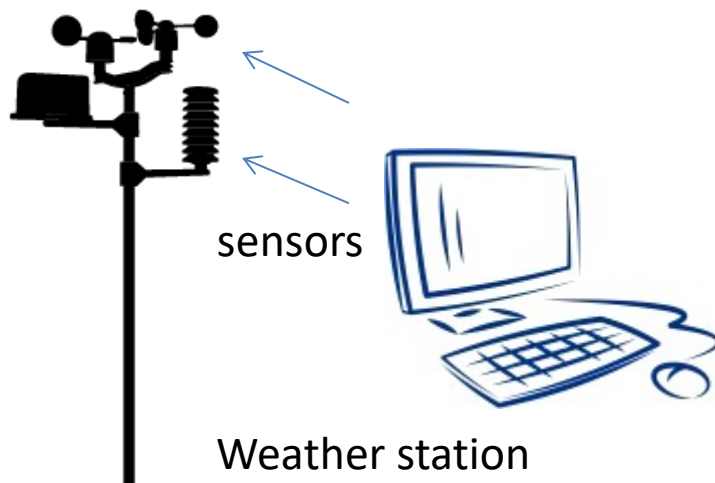
Strategy Pattern Description (4)

- ▶ *Communication overhead between Strategy and Context.*
The Strategy interface is shared by all Concrete Strategy classes whether the algorithms they implement are trivial or complex.
 - ▶ Hence it's likely that some Concrete Strategies won't use all the information passed to them through this interface. That means there will be times when the context creates and initializes parameters that never get used
- ▶ *Increased number of objects.*
Strategies increase the number of objects in an application.

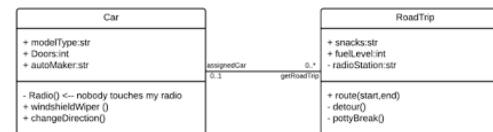
Design Pattern: Observer

- ▶ The Observer Pattern is one of the most heavily used patterns in JDK:
 - ▶ It is a pattern that keeps your objects in the know when something they might care about happens.
 - ▶ Objects can even decide at runtime whether they want to be kept informed.

▶ Motivation : the weather station



WeatherData Object



Display Devices



pulls data

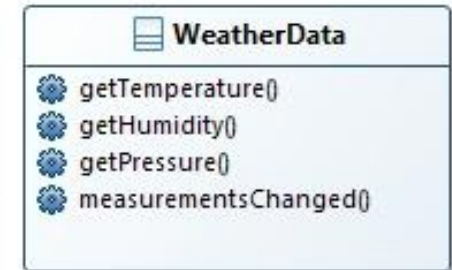
displays

The same data has to be represented in different ways by different devices

Motivation

- ▶ The WeatherData object talks to the weather station and is used to update three display devices (current conditions weather statistics and forecast).

The measurementsChanged() method is called any time new weather measurement data is available



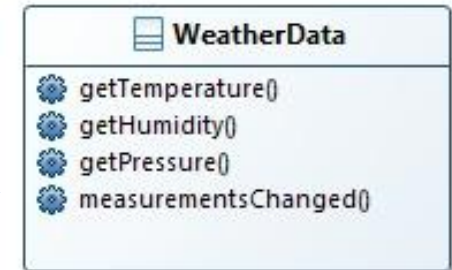
We need to implement three display elements that use the weather data, and make possible to add new displays

```
public class WeatherData {  
    // instance variable declarations  
    public void measurementsChanged() {  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        Display1.update(temp, humidity, pressure);  
        Display2.update(temp, humidity, pressure);  
        Display3.update(temp, humidity, pressure);  
    }  
    // other WeatherData methods here  
}
```

Motivation

- ▶ The WeatherData object talks to the weather station and is used to update three display devices (current conditions weather statistics and forecast).

The measurementsChanged() method is called any time new weather measurement data is available



We need to implement three display elements that use the weather data, and make possible to add new displays

```
public class WeatherData {  
    // instance variable declarations  
    public void measurementsChanged() {  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        Display1.update(temp, humidity, pressure);  
        Display2.update(temp, humidity, pressure);  
        Display3.update(temp, humidity, pressure);  
    }  
    // other WeatherData methods here  
}
```

ERROR!

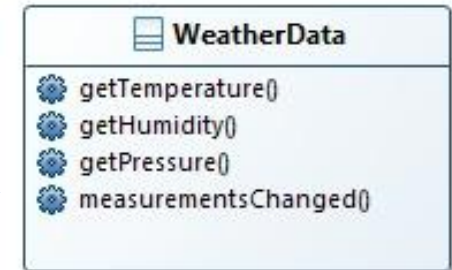
We are coding the concrete implementation!

No way to change it without changing the software application

Motivation

- ▶ The WeatherData object talks to the weather station and is used to update three display devices (current conditions weather statistics and forecast).

The measurementsChanged() method is called any time new weather measurement data is available



We need to implement three display elements that use the weather data, and make possible to add new displays

```
public class WeatherData {  
    // instance variable declarations  
    public void measurementsChanged() {  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        Display1.update(temp, humidity, pressure);  
        Display2.update(temp, humidity, pressure);  
        Display3.update(temp, humidity, pressure);  
    }  
    // other WeatherData methods here  
}
```

AREA OF CHANGE

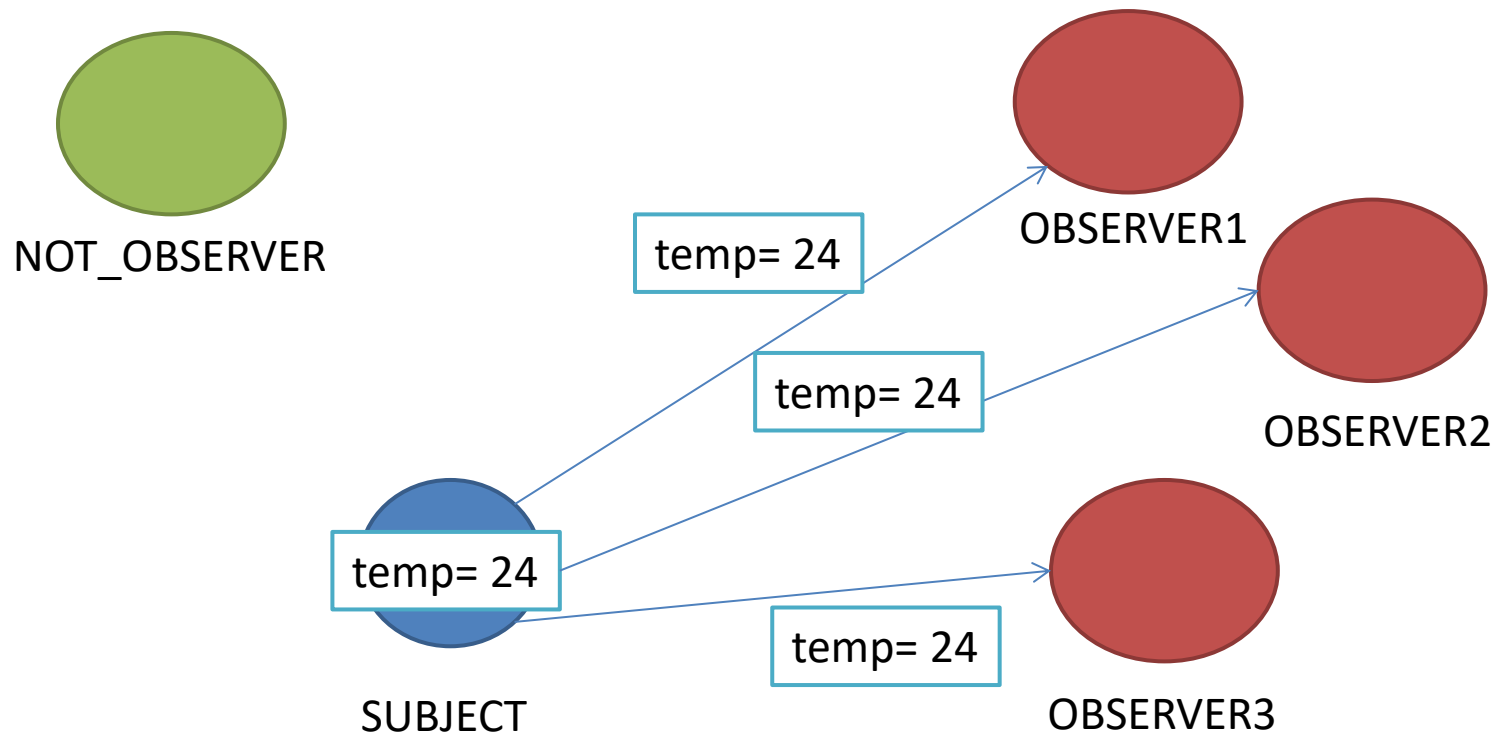
We need to encapsulate it!!!

The publisher - subscriber mechanism

- ▶ Think about a news paper
 - ▶ Newspapers are online and publish articles
 - ▶ You subscribe to a particular newspaper and when there is a new edition, it gets delivered to you
 - ▶ You unsubscribe when you don't want papers anymore, and they stop to be delivered
 - ▶ People can subscribe / unsubscribe dynamically

Publisher + Subscribers = Observer Pattern

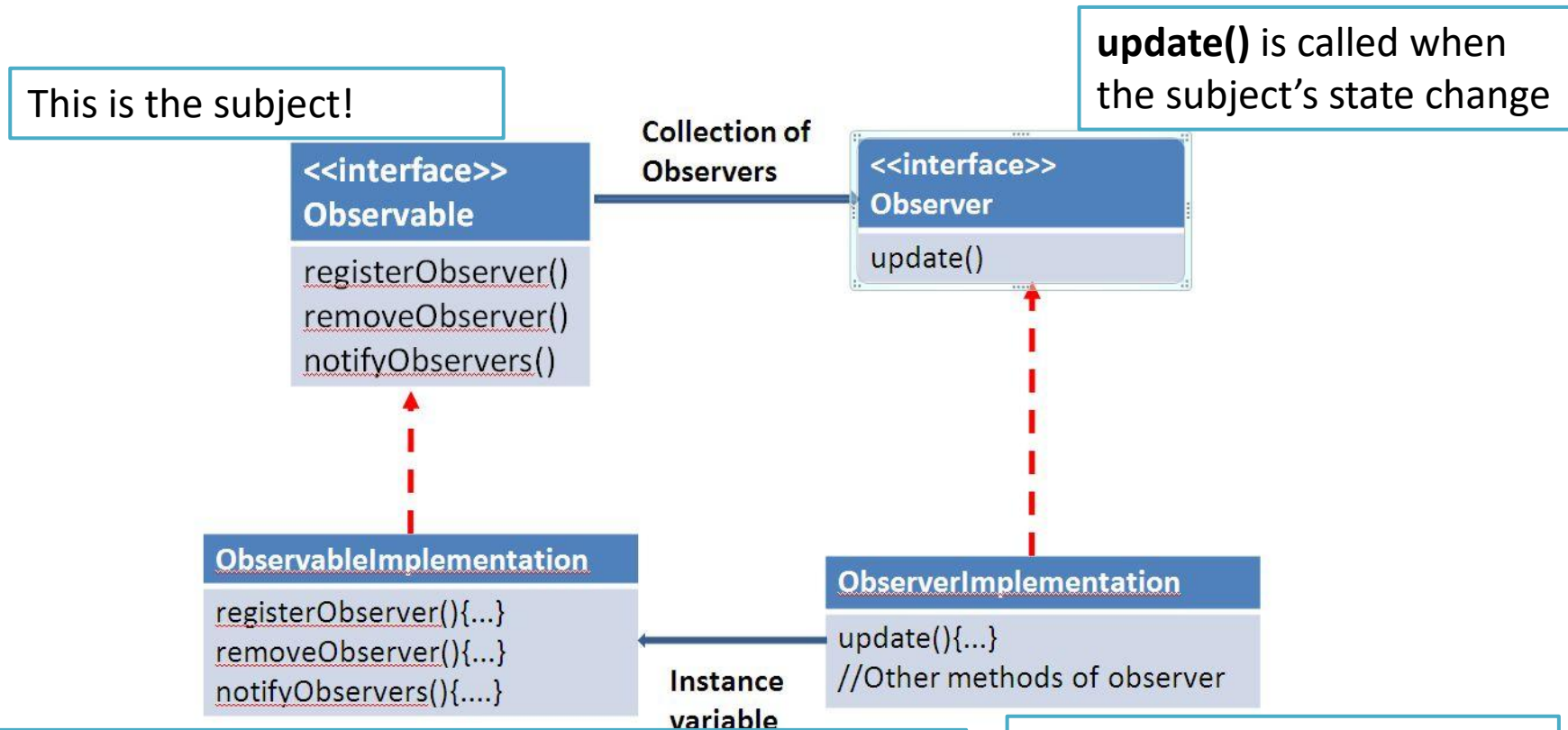
- Actors: the SUBJECT provides the data to the OBSERVERS.



Actually, the mechanism to transfer the data can be based on pull/push architecture

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

The Class Diagram



A concrete subject implements the Subject interface. The concrete subject implements a `notifyObservers()` method to update all the current observers whenever state changes

Concrete observers can be any class that implements the Observer interface. Each observer registers with a concrete subject to receive updates.

Loose Coupling

- ▶ **Design Principle**
Strive for loosely coupled designs between objects that interact.

Loosely coupled designs allow us to build flexible OO systems that can handle change because they minimize the interdependency between objects

- ▶ The Observer Pattern provides an object design where subjects and observers are loosely coupled.
 - ▶ The only thing the subject knows about an observer is that it implements a certain interface (the Observer interface).
 - ▶ We can add, replace and remove new observers at any time.
 - ▶ We never need to modify the subject to add new types of observers.
 - ▶ We can reuse subjects or observers independently of each other.
 - ▶ If we have another use for a subject or an observer, we can easily reuse them because the two aren't tightly coupled.
 - ▶ Changes to either the subject or an observer will not affect the other.

Implementing the Weather Station



```
public interface Subject {
    public void registerObserver(Observer o);
    public void removeObserver(Observer o);
    public void notifyObservers();
}

public interface Observer {
    public void update(float temp, float humidity, float
pressure);
}

public interface DisplayElement {
    public void display();
}
```


Implementing the Weather Station (2)

```
public class WeatherData implements Subject {
    private ArrayList<Observer> observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList<Observer>();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        int i = observers.indexOf(o);
        if (i >= 0) {
            observers.remove(i);
        }
    }

    public void notifyObservers() {
        for (Observer observer : observers)
        {
            observer.update(temperature,
                humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float
        temperature, float humidity, float
        pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    // other WeatherData methods here
}
```

Implementing the Weather Station (3)

```
public class Display1 implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private Subject weatherData;

    public Display1(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

Implementing the Weather Station (4)

```
public class WeatherStation {  
    public static void main(String[] args) {  
        WeatherData weatherData = new WeatherData();  
  
        Display1 currentDisplay = new Display1(weatherData);  
  
        weatherData.setMeasurements(80, 65, 30.4f);  
        weatherData.setMeasurements(82, 70, 29.2f);  
        weatherData.setMeasurements(78, 90, 29.2f);  
    }  
}
```

Push and Pull mechanism

- ▶ It is always the responsibility of Observable object to notify all the subscribed observers, but the difference lies whether
 - ▶ [Push->] the observer gets the exact data it wants
 - ▶ [Pull->] it gets the data wrapped in some object (mostly Observable object) & it has to extract the required data from it.

Push Mechanism

- ▶ This is purely the Observable's responsibility, Observer just need to make sure they have put required code in their update methods.
 - ▶ **Advantages**
 - ▶ The main advantage is lower coupling between the observer and the subject.
 - ▶ Observable & Observer both are interfaces/abstract classes which is actually a design principle - **Program to interface or supertypes**
 - ▶ **Disadvantage**
 - ▶ Less flexibility : As Observable needs to send the required data to the Observer and it would become messy if we have say 1000 observers and most of them require different types of data.
 - ▶ **Use-Case**
 - ▶ It should be used when there are max 2-3 different types of Observers (different types means observer require different data) or all observers require same type of data.

Pull Mechanism

- ▶ It is the Observable's responsibility to notify all observer that something has been changed
 - ▶ The Observable shares the whole object which has the changes
 - ▶ Observers extract the required details from that complete project in their update methods.
 - ▶ Advantages
 - ▶ More flexibility: each observer can decide for itself what to query, without relying on the subject to send the correct (only required) information.
 - ▶ Disadvantage
 - ▶ The observers would have to know things about the subject in order to query the right information from the shared complete object.
 - ▶ Use-Case
 - ▶ It should be used when there are more than 2-3 different types of Observers (different types means observer require different data)

Key Points

- ▶ The Observer Pattern defines a one-to-many relationship between objects.
- ▶ Subjects (Observables) update Observers using a common interface.
- ▶ Observers are loosely coupled in that the Observable knows nothing about them, other than that they implement the Observer interface.
- ▶ You can push or pull data from the Observable when using the pattern.
- ▶ Don't depend on a specific order of notification for your Observers.
- ▶ Java has several implementations of the Observer Pattern, including the general purpose `java.util.Observable`.
- ▶ Watch out for issues with the `java.util.Observable` implementation.
- ▶ Don't be afraid to create your own Observable implementation if needed.
- ▶ Swing makes heavy use of the Observer Pattern, as do many GUI frameworks.
 - ▶ You'll also find the pattern in many other places, including JavaBeans and RMI.

Observer Pattern Description - GoF structure (partial)

▶ Intent

- ▶ Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

▶ Also Known As

- ▶ Dependents, Publish-Subscribe

▶ Applicability

- ▶ Use the Observer pattern in any of the following situations:
 - ▶ When an abstraction has two aspects, one dependent on the other. Encapsulating these aspects in separate objects lets you vary and reuse them independently.
 - ▶ When a change to one object requires changing others, and you don't know how many objects need to be changed.
 - ▶ When an object should be able to notify other objects without making assumptions about who these objects are. In other words, you don't want these objects tightly coupled.

► Structure

► Participants

► Subject

- knows its observers. Any number of Observer objects may observe a subject.
- provides an interface for attaching and detaching Observer objects.

► Observer

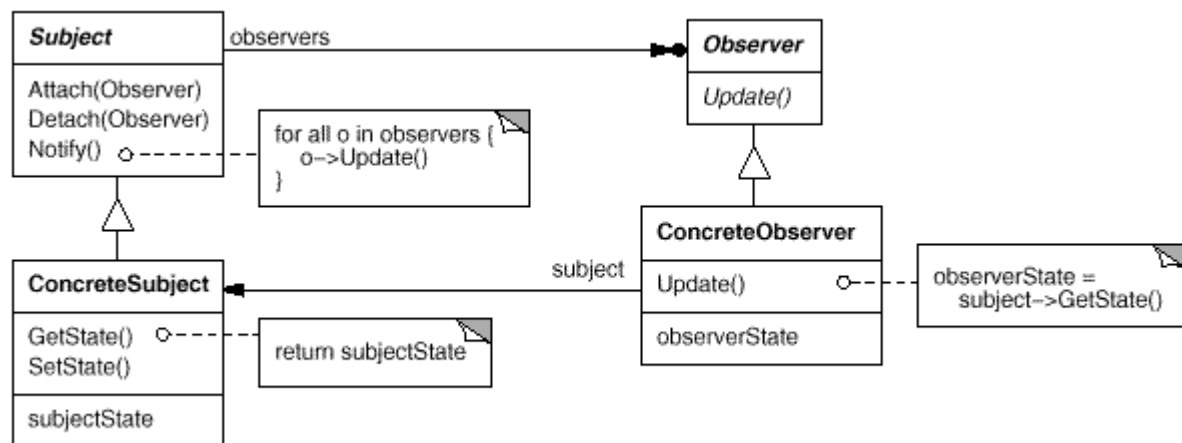
- defines an updating interface for objects that should be notified of changes in a subject.

► ConcreteSubject

- stores state of interest to ConcreteObserver objects and sends a notification to its observers when its state changes.

► ConcreteObserver

- maintains a reference to a ConcreteSubject object.
- stores state that should stay consistent with the subject's.
- implements the Observer updating



Observer Pattern Description(3)

► Consequences

- *Abstract coupling between Subject and Observer.*
- *Support for broadcast communication.*
 - Unlike an ordinary request, the notification that a subject sends needn't specify its receiver. The notification is broadcast automatically to all interested objects that subscribed to it.
- *Unexpected updates.*
 - Because observers have no knowledge of each other's presence, they can be blind to the ultimate cost of changing the subject.



References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, Second Edition, O'Reilly Media 2020
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DELL'insegnamento di Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.